Part B — Project Work (Hour 2)

MiniBank Report

CONCEPTS USED

Practical 1: Class, object, record, enum, Scanner, switch, loops.

Practical 2: Encapsulation, constructors, static, final, getters.

Practical 3: toString(), equals(), hashCode(), cloning, nested class, instanceof.

Practical 4: Strings, regex, input validation, command parsing, StringBuilder.

Practical 5: Inheritance, abstraction, polymorphism, method overriding, super.

Practical 6: Interfaces, default methods, functional and marker interfaces, lambdas.

Practical 7: Custom annotations, reflection, @Override, @Retention, @Target.

Practical 8: Exception handling, custom exceptions, try-catch-finally, throw, throws.

Practical 9: Multithreading, Runnable, synchronization, race conditions.

Practical 10: ExecutorService, thread pools, task submission, shutdown.

Practical 11: File handling, serialization, deserialization, persistence, logging.

Practical 12: ConcurrentHashMap, collections, thread safety, sorting, comparators.

Practical 13: Generics and HashMap

Practical 14: Service layer, banking operations, transactions, reports, integration, batch processing.

WORKING

The MiniBank application starts by displaying the bank name and a menu of available operations. The user can create a savings, current, or fixed deposit account by entering the required details. Deposits increase the account balance, while withdrawals follow the rules of the selected account type. The application also supports transferring money between accounts, generating account statements, listing accounts sorted by balance, viewing transaction logs, and processing batch deposits. Account information is saved to a file when the application exits and loaded again when it restarts. Exception handling manages invalid amounts, insufficient funds, missing accounts, and incorrect inputs, while synchronization and concurrent collections support safe access to shared data during concurrent operations.

CONCLUSION

The MiniBank project successfully demonstrates the implementation of a console-based banking application using Java. It supports essential operations such as account creation, deposits, withdrawals, fund transfers, account statements, and transaction logging. Through this project, various Object-Oriented Programming concepts, including encapsulation, inheritance, abstraction, polymorphism, interfaces, exception handling, generics, multithreading, and file handling, were implemented. The project also provides experience in data persistence, input validation, and modular program design. Overall, this practical helped strengthen my understanding of Java programming and the development of structured, reliable applications.

GITHUB REPO LINK

https://github.com/dharmik1d3a/Java_Practical

CODE EXECUTION AND OUTPUT

PROBLEM	All the parts now exist. In this final capstone lab you assemble MiniBank and finish it to a deliverable standard: a BankService wires the repository, validation and exceptions together and the Practical 1 menu calls real operations; you add safe batch processing and the report; and you test and package the application. This is the Milestone 3 final checkpoint and viva. 1. Create a class named BankService that uses Repository, the Validator and the exception classes to provide openAccount, deposit, withdraw, transfer, statement and listAccounts. 2. Connect the Practical 1 menu so each option calls the matching BankService method; load saved accounts on start-up (Practical 11) and save them on exit. 3. Use the TransactionProcessor (Practical 10) over the thread-safe store (Practical 12) to process a batch of transactions safely, and add the report (Practical 11) as a menu option. 4. Write a few JUnit tests using @Test: a deposit increases the balance; a withdrawal over the limit throws an exception; many concurrent deposits keep the balance correct. 5. (Optional) Add an @Audit annotation and a reflective logger that records which service method was called. 1. 6. Build the final runnable JAR and complete the README.

CODE

```
import java.io.*;
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.regex.Pattern;

public class MiniBank {
   record BankInfo(String name, String branch) {
   }

   enum MenuOption {
      OPEN_ACCOUNT, DEPOSIT, WITHDRAW, TRANSFER, EXIT
   }

   @Retention(RetentionPolicy.RUNTIME)
   @Target(ElementType.FIELD)
   @interface Id {
   }

   @Retention(RetentionPolicy.RUNTIME)
   @Target(ElementType.FIELD)
   @interface Positive {
      String message() default "must be > 0";
   }

   @Retention(RetentionPolicy.RUNTIME)
   @Target(ElementType.FIELD)
   @interface MaxLength {
      int value();
   }

   interface Transactable {
      void deposit(long amount) throws BankException;

      boolean withdraw(long amount) throws BankException;
   }

   interface InterestBearing {
      double interestRate();

      default double yearlyInterest(long balance) {
         return balance * interestRate() / 100.0;
      }
   }
```

```
interface Premium {
}

@FunctionalInterface
interface WithdrawRule {
   boolean allow(Account a, long amount);
}

enum TransactionType {
   DEPOSIT, WITHDRAW, TRANSFER
}

record Command(TransactionType type, String accountNumber, long amount) {
}

static class BankException extends Exception {
   BankException(String m) {
      super(m);
   }
}

static class InsufficientFundsException extends BankException {
   final long shortfall;

   InsufficientFundsException(long s) {
      super("Insufficient funds; short by " + s);
      shortfall = s;
   }
}

static class AccountNotFoundException extends BankException {
   AccountNotFoundException(String n) {
      super("Account not found: " + n);
   }
}

static class InvalidAmountException extends BankException {
   InvalidAmountException() {
      super("Amount must be positive");
   }
}

static class Customer implements Serializable, Cloneable {
   private static final long serialVersionUID = 1L;
   private static long counter = 100;
   @MaxLength(60)
   private final String name;
   private final String email, mobile;
```

```
    private final String customerId;
    private Address address;

    static class Address implements Serializable {
        private static final long serialVersionUID = 1L;
        private final String line, city, pincode;

        Address(String l, String c, String p) {
            line = l;
            city = c;
            pincode = p;
        }

        public String getLine() {
            return line;
        }

        public String getCity() {
            return city;
        }

        public String getPincode() {
            return pincode;
        }
    }

    Customer(String n, String e, String m, Address a) {
        name = n;
        email = e;
        mobile = m;
        address = a;
        customerId = "CUST" + (++counter);
    }

    public String getName() {
        return name;
    }

    public String getEmail() {
        return email;
    }

    public String getMobile() {
        return mobile;
    }

    public String getCustomerId() {
        return customerId;
```

```
    }

    public Address getAddress() {
      return address;
    }

    @Override
    public Customer clone() {
      try {
        return (Customer) super.clone();
      } catch (CloneNotSupportedException e) {
        throw new AssertionError(e);
      }
    }
  }

  static abstract class Account implements Transactable, InterestBearing, Serializable,
Comparable<Account> {
    private static final long serialVersionUID = 1L;
    private static long counter = 0;
    @Id
    private final String accountNumber;
    @MaxLength(60)
    private String ownerName;
    @Positive
    private long balance;
    private boolean active = true;

    Account(String owner, long opening) {
      accountNumber = String.format("AC%04d", ++counter);
      ownerName = owner;
      balance = opening;
    }

    public String getAccountNumber() {
      return accountNumber;
    }

    public String getOwnerName() {
      return ownerName;
    }

    public synchronized long getBalance() {
      return balance;
    }

    public boolean isActive() {
      return active;
```

```
    }

    public void close() {
      active = false;
    }

    protected synchronized void changeBalance(long delta) {
      balance += delta;
    }

    public synchronized void deposit(long amount) throws BankException {
      if (amount <= 0)
        throw new InvalidAmountException();
      if (!active)
        throw new BankException("Account is inactive");
      changeBalance(amount);
    }

    public synchronized boolean withdraw(long amount) throws BankException {
      if (amount <= 0)
        throw new InvalidAmountException();
      if (!active)
        throw new BankException("Account is inactive");
      if (!canWithdraw(amount))
        throw new InsufficientFundsException(Math.max(0, amount - balance));
      changeBalance(-amount);
      return true;
    }

    abstract boolean canWithdraw(long amount);

    @Override
    public int compareTo(Account a) {
      return accountNumber.compareTo(a.accountNumber);
    }

    @Override
    public boolean equals(Object o) {
      return o instanceof Account a && accountNumber.equals(a.accountNumber);
    }

    @Override
    public int hashCode() {
      return Objects.hash(accountNumber);
    }

    @Override
    public String toString() {
```

```
        return accountNumber + " | " + ownerName + " | Balance: " + balance + " | Active: " +
active;
      }
   }

   static class SavingsAccount extends Account {
      private static final long serialVersionUID = 1L;
      private final long minBalance;

      SavingsAccount(String n, long b, long min) {
         super(n, b);
         minBalance = min;
      }

      @Override
      public double interestRate() {
         return 4.0;
      }

      @Override
      boolean canWithdraw(long a) {
         return getBalance() - a >= minBalance;
      }
   }

   static class CurrentAccount extends Account {
      private static final long serialVersionUID = 1L;
      private final long overdraftLimit;

      CurrentAccount(String n, long b, long limit) {
         super(n, b);
         overdraftLimit = limit;
      }

      @Override
      public double interestRate() {
         return 0.0;
      }

      @Override
      boolean canWithdraw(long a) {
         return getBalance() - a >= -overdraftLimit;
      }

      @Override
      public synchronized boolean withdraw(long a) throws BankException {
         if (a <= 0)
            throw new InvalidAmountException();
```

```
        if (getBalance() - a < -overdraftLimit)
          throw new InsufficientFundsException(a - getBalance());
        changeBalance(-a);
        return true;
      }
    }

    static class FixedDepositAccount extends Account {
      private static final long serialVersionUID = 1L;

      FixedDepositAccount(String n, long b) {
        super(n, b);
      }

      @Override
      public double interestRate() {
        return 7.0;
      }

      @Override
      boolean canWithdraw(long a) {
        return false;
      }
    }

    static class Validator {
      static boolean isValidMobile(String s) {
        return s != null && s.matches("[6-9][0-9]{9}");
      }

      static boolean isValidEmail(String s) {
        return s != null && Pattern.matches("[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}",
s);
      }

      static boolean isValidPan(String s) {
        return s != null && s.matches("[A-Z]{5}[0-9]{4}[A-Z]");
      }

      static boolean isValidIfsc(String s) {
        return s != null && s.matches("[A-Z]{4}0[A-Z0-9]{6}");
      }
    }

    static class CommandParser {
      static Command parse(String line) {
        String[] p = line.trim().split("\\s+");
        if (p.length != 3)
```

```
            throw new IllegalArgumentException("Use: TYPE ACCOUNT AMOUNT");
        return new Command(TransactionType.valueOf(p[0].toUpperCase()), p[1],
Long.parseLong(p[2]));
      }
   }

   static class StatementFormatter {
      static String buildStatement(Account a) {
         return new StringBuilder("MINIBANK STATEMENT\nAccount:
").append(a.getAccountNumber()).append("\nOwner: ")
               .append(a.getOwnerName()).append("\nBalance:
₹").append(a.getBalance()).append("\nInterest rate: ")
               .append(a.interestRate()).append("%\n").toString();
      }
   }

   static class AnnotationValidator {
      static String[] validate(Object o) {
         List<String> errors = new ArrayList<>();
         for (Field f : o.getClass().getDeclaredFields()) {
            try {
               f.setAccessible(true);
               Object v = f.get(o);
               if (f.isAnnotationPresent(Positive.class) && v instanceof Number n &&
n.longValue() <= 0)
                  errors.add(f.getName() + ": " + f.getAnnotation(Positive.class).message());
               if (f.isAnnotationPresent(MaxLength.class) && v instanceof String s
                     && s.length() > f.getAnnotation(MaxLength.class).value())
                  errors.add(f.getName() + ": too long");
            } catch (ReflectiveOperationException e) {
               errors.add(e.getMessage());
            }
         }
         return errors.toArray(new String[0]);
      }
   }

   static class Repository<T> {
      private final HashMap<String, T> data = new HashMap<>();

      private String idOf(T x) {
         try {
            // Search this class and its superclasses for the field marked @Id.
            Class<?> type = x.getClass();
            while (type != null) {
               for (Field f : type.getDeclaredFields()) {
                  if (f.isAnnotationPresent(Id.class)) {
                     f.setAccessible(true);
```

```
                    return String.valueOf(f.get(x));
                  }
                }
                type = type.getSuperclass();
              }
              throw new IllegalArgumentException("No @Id field in " +
x.getClass().getSimpleName());
            } catch (IllegalAccessException e) {
              throw new IllegalStateException(e);
            }
          }

          T save(T x) {
            data.put(idOf(x), x);
            return x;
          }

          T findById(String id) {
            return data.get(id);
          }

          ArrayList<T> findAll() {
            return new ArrayList<>(data.values());
          }

          boolean delete(String id) {
            return data.remove(id) != null;
          }

          @SafeVarargs
          final void saveAll(T... xs) {
            for (T x : xs)
              save(x);
          }

          static <N extends Number> double average(N[] a) {
            return Arrays.stream(a).mapToDouble(Number::doubleValue).average().orElse(0);
          }
        }

        static class AccountStore {
          private final ConcurrentHashMap<String, Account> accounts = new
ConcurrentHashMap<>();
          private final List<String> history = Collections.synchronizedList(new ArrayList<>());

          void add(Account a) {
            accounts.put(a.getAccountNumber(), a);
          }
```

```
    Account get(String id) {
        return accounts.get(id);
    }

    Collection<Account> all() {
        return accounts.values();
    }

    ArrayList<Account> sortedByBalance() {
        ArrayList<Account> a = new ArrayList<>(accounts.values());
        a.sort(Comparator.comparingLong(Account::getBalance));
        return a;
    }

    void log(String s) {
        history.add(java.time.LocalDateTime.now() + " | " + s);
    }

    List<String> history() {
        return history;
    }
}

static class AccountWorker implements Runnable {
    private final Account account;
    private final int times;
    private final long amount;

    AccountWorker(Account a, int t, long m) {
        account = a;
        times = t;
        amount = m;
    }

    public void run() {
        for (int i = 0; i < times; i++)
            try {
                account.deposit(amount);
            } catch (BankException e) {
                System.out.println(e.getMessage());
            }
    }
}

static class TransactionProcessor implements AutoCloseable {
    private final ExecutorService pool = Executors.newFixedThreadPool(3);
```

```
    void deposit(Account a, long amount) {
        pool.submit(() -> {
            try {
                a.deposit(amount);
            } catch (BankException e) {
                System.out.println(e.getMessage());
            }
        });
    }

    void process(List<Runnable> jobs) {
        for (Runnable r : jobs)
            pool.submit(r);
        pool.shutdown();
        try {
            if (!pool.awaitTermination(10, TimeUnit.SECONDS))
                pool.shutdownNow();
        } catch (InterruptedException e) {
            pool.shutdownNow();
            Thread.currentThread().interrupt();
        }
    }

    public void close() {
        pool.shutdown();
    }
  }

  static class StatePersister {
    static void save(Account[] a, Path p) throws IOException {
        try (ObjectOutputStream out = new ObjectOutputStream(Files.newOutputStream(p)))
{
            out.writeObject(a);
        }
    }

    static Account[] load(Path p) throws IOException, ClassNotFoundException {
        if (!Files.exists(p))
            return new Account[0];
        try (ObjectInputStream in = new ObjectInputStream(Files.newInputStream(p))) {
            return (Account[]) in.readObject();
        }
    }
  }

  static class TransactionLog {
    static void append(Path p, String s) throws IOException {
        Files.createDirectories(p.getParent());
```

```
            Files.writeString(p, s + System.lineSeparator(), StandardCharsets.UTF_8,
StandardOpenOption.CREATE,
                StandardOpenOption.APPEND);
        }
    }

    static class ReportGenerator {
        static void report(Path dir) throws IOException {
            if (!Files.exists(dir)) {
                System.out.println("No reports yet.");
                return;
            }
            try (DirectoryStream<Path> ds = Files.newDirectoryStream(dir)) {
                for (Path p : ds)
                    if (Files.isRegularFile(p)) {
                        System.out.println("-- " + p.getFileName() + " (" + Files.size(p) + " bytes) --");
                        try (BufferedReader r = Files.newBufferedReader(p)) {
                            r.lines().forEach(System.out::println);
                        }
                    }
            }
        }
    }

    static class BankService {
        private final Repository<Account> repo = new Repository<>();
        private final AccountStore store = new AccountStore();
        private final Path state = Path.of("accounts.dat"), log = Path.of("logs",
"transactions.log");

        BankService() {
            try {
                for (Account a : StatePersister.load(state)) {
                    repo.save(a);
                    store.add(a);
                }
            } catch (Exception e) {
                System.out.println("Starting with empty data: " + e.getMessage());
            }
        }

        Account openAccount(String name, long opening, int type) throws BankException {
            if (opening < 0)
                throw new InvalidAmountException();
            Account a = switch (type) {
                case 1 -> new SavingsAccount(name, opening, 500);
                case 2 -> new CurrentAccount(name, opening, 5000);
                default -> new FixedDepositAccount(name, opening);
```

```
        };
        repo.save(a);
        store.add(a);
        record("OPEN " + a.getAccountNumber() + " " + opening);
        return a;
    }

    Account find(String id) throws AccountNotFoundException {
        Account a = repo.findById(id);
        if (a == null)
            throw new AccountNotFoundException(id);
        return a;
    }

    void deposit(String id, long amount) throws BankException {
        Account a = find(id);
        a.deposit(amount);
        record("DEPOSIT " + id + " " + amount);
    }

    void withdraw(String id, long amount) throws BankException {
        Account a = find(id);
        a.withdraw(amount);
        record("WITHDRAW " + id + " " + amount);
    }

    void transfer(String from, String to, long amount) throws BankException {
        Account a = find(from), b = find(to);
        if (a == b)
            throw new BankException("Choose two different accounts");
        a.withdraw(amount);
        try {
            b.deposit(amount);
        } catch (BankException e) {
            try {
                a.deposit(amount);
            } catch (BankException ignored) {
            }
            throw e;
        }
        record("TRANSFER " + from + " -> " + to + " " + amount);
    }

    void statement(String id) throws BankException {
        System.out.println(StatementFormatter.buildStatement(find(id)));
    }

    void listAccounts() {
```

```
        for (Account a : store.sortedByBalance())
            System.out.println(a);
    }

    void report() {
        try {
            ReportGenerator.report(Path.of("logs"));
        } catch (IOException e) {
            System.out.println("Report error: " + e.getMessage());
        }
    }

    void batch() {
        List<Runnable> tasks = new ArrayList<>();
        for (Account a : store.all())
            tasks.add(() -> {
                try {
                    a.deposit(1);
                } catch (BankException ignored) {
                }
            });
        try (TransactionProcessor p = new TransactionProcessor()) {
            p.process(tasks);
        }
        System.out.println("Batch completed.");
    }

    void save() {
        try {
            StatePersister.save(store.all().toArray(new Account[0]), state);
        } catch (IOException e) {
            System.out.println("Could not save: " + e.getMessage());
        }
    }

    private void record(String s) {
        store.log(s);
        try {
            TransactionLog.append(log, s);
        } catch (IOException e) {
            System.out.println("Log warning: " + e.getMessage());
        }
    }
}

public static void main(String[] args) {
    System.out.println(new BankInfo("MiniBank", "Charusat Branch"));
    BankService bank = new BankService();
```

```
        Scanner sc = new Scanner(System.in);
        boolean running = true;
        while (running) {
          System.out.println(
                "\n1.Open account  2.Deposit  3.Withdraw  4.Transfer  5.Statement  6.List
accounts  7.Report  8.Batch deposit  9.Exit");
          System.out.print("Choice: ");
          String raw = sc.nextLine().trim();
          int ch;
          try {
            ch = Integer.parseInt(raw);
          } catch (NumberFormatException e) {
            System.out.println("Enter a number.");
            continue;
          }
          try {
            switch (ch) {
              case 1 -> {
                System.out.print("Owner name: ");
                String n = sc.nextLine();
                System.out.print("Opening balance: ");
                long b = Long.parseLong(sc.nextLine());
                System.out.print("Type (1 Savings, 2 Current, 3 Fixed Deposit): ");
                int t = Integer.parseInt(sc.nextLine());
                System.out.println("Created: " + bank.openAccount(n, b, t));
              }
              case 2 -> {
                System.out.print("Account number: ");
                String id = sc.nextLine();
                System.out.print("Amount: ");
                bank.deposit(id, Long.parseLong(sc.nextLine()));
                System.out.println("Deposit successful.");
              }
              case 3 -> {
                System.out.print("Account number: ");
                String id = sc.nextLine();
                System.out.print("Amount: ");
                bank.withdraw(id, Long.parseLong(sc.nextLine()));
                System.out.println("Withdrawal successful.");
              }
              case 4 -> {
                System.out.print("From account: ");
                String a = sc.nextLine();
                System.out.print("To account: ");
                String b = sc.nextLine();
                System.out.print("Amount: ");
                bank.transfer(a, b, Long.parseLong(sc.nextLine()));
                System.out.println("Transfer successful.");
```

```
                }
                case 5 -> {
                    System.out.print("Account number: ");
                    bank.statement(sc.nextLine());
                }
                case 6 -> bank.listAccounts();
                case 7 -> bank.report();
                case 8 -> bank.batch();
                case 9 -> {
                    bank.save();
                    running = false;
                    System.out.println("Goodbye!");
                }
                default -> System.out.println("Invalid choice.");
            }
        } catch (NumberFormatException e) {
            System.out.println("Enter a valid number.");
        } catch (BankException e) {
            System.out.println("Operation failed: " + e.getMessage());
        } finally {
            System.out.println("Operation handled.");
        }
    }
    bank.save();
    sc.close();
  }
}
```

OUTPUT

```
BankInfo[name=MiniBank, branch=Charusat Branch]

1.Open account  2.Deposit  3.Withdraw  4.Transfer  5.Statement  6.List accounts  7.Report  8.Batch deposit  9.Exit
Choice: 1
Owner name: JALPESH VASA
Opening balance: 5000000
Type (1 Savings, 2 Current, 3 Fixed Deposit): 1
Created: AC0001 | JALPESH VASA | Balance: 5000000 | Active: true
Operation handled.

1.Open account  2.Deposit  3.Withdraw  4.Transfer  5.Statement  6.List accounts  7.Report  8.Batch deposit  9.Exit
Choice: 2
Account number: AC0001
Amount: 5000000
Deposit successful.
Operation handled.

1.Open account  2.Deposit  3.Withdraw  4.Transfer  5.Statement  6.List accounts  7.Report  8.Batch deposit  9.Exit
Choice: 1
Owner name: OM SUTARIYA
Opening balance: 0
Type (1 Savings, 2 Current, 3 Fixed Deposit): 1
Created: AC0002 | OM SUTARIYA | Balance: 0 | Active: true
Operation handled.

1.Open account  2.Deposit  3.Withdraw  4.Transfer  5.Statement  6.List accounts  7.Report  8.Batch deposit  9.Exit
Choice: 4
From account: AC0001
To account: AC0002
Amount: 500000
Transfer successful.
Operation handled.

1.Open account  2.Deposit  3.Withdraw  4.Transfer  5.Statement  6.List accounts  7.Report  8.Batch deposit  9.Exit
Choice: 5
Account number: AC0001
MINIBANK STATEMENT
Account: AC0001
Owner: JALPESH VASA
Balance: ?9500000
Interest rate: 4.0%

Operation handled.
```

```
1.Open account  2.Deposit  3.Withdraw  4.Transfer  5.Statement  6.List accounts  7.Report  8.Batch deposit  9.Exit
Choice: 6
AC0002 | OM SUTARIYA | Balance: 500000 | Active: true
AC0001 | JALPESH VASA | Balance: 9500000 | Active: true
Operation handled.

1.Open account  2.Deposit  3.Withdraw  4.Transfer  5.Statement  6.List accounts  7.Report  8.Batch deposit  9.Exit
Choice: 7
-- transactions.log (167 bytes) --
OPEN AC0001 5000000
DEPOSIT AC0001 500000000
WITHDRAW AC0001 50000000
OPEN AC0001 5000000
DEPOSIT AC0001 5000000
OPEN AC0002 0
TRANSFER AC0001 -> AC0002 500000
Operation handled.

1.Open account  2.Deposit  3.Withdraw  4.Transfer  5.Statement  6.List accounts  7.Report  8.Batch deposit  9.Exit
Choice: 8
Batch completed.
Operation handled.

1.Open account  2.Deposit  3.Withdraw  4.Transfer  5.Statement  6.List accounts  7.Report  8.Batch deposit  9.Exit
Choice: 9
Goodbye!
Operation handled.
PS C:\Users\aimlo\OneDrive\Desktop\MiniBank_14_Cumulative_Practicals\minibank_practicals\Practical_14>
```